

# Spécification Technique Synthétique pour l'Application de Gestion Client/Assurance INTIA

Ce document énonce la spécification technique pour le développement d'une application web de gestion de clients et de polices d'assurance pour INTIA Assurance.

## I. Analyse Fonctionnelle et Modèle d'Acteurs

Le système est conçu pour gérer trois types d'acteurs principaux, chacun ayant un rôle bien défini :

1. **Direction Générale (Rôle : Administrateur / Admin)**
  - **Objectif** : Supervision et gestion centralisée des succursales.
  - **Permissions** : Gestion complète des succursales (CRUD) et consultation intégrale de tous les clients et polices du groupe.
2. **Succursale (Rôle : Manager / Manager)**
  - **Objectif** : Enregistrement et gestion des clients rattachés à son département géographique.
  - **Permissions** : Gestion complète des clients et polices de son portefeuille (CRUD).
3. **Client (Rôle : Utilisateur / Client)**
  - **Objectif** : Consultation de ses informations personnelles et de ses contrats d'assurance.
  - **Permissions** : Consultation de son propre profil, demande de modification ou de suppression de données (soumise à validation).

## II. Modèle de Fonctionnement et Interactions

### Direction Générale (Admin)

- **Actions Principales** : Accès à la liste de toutes les succursales ; Création, mise à jour et suppression des succursales ; Visualisation de l'intégralité des clients enregistrés par chaque succursale.
- **Contraintes** : Aucune restriction sur les données de l'entreprise.

### Succursale (Manager)

- **Actions Principales** : Création, mise à jour et suppression des fiches client ; Visualisation intégrale de la liste des clients de son département ; Gestion des polices d'assurance associées.
- **Contraintes** : Limitation stricte aux données des clients rattachés à son centre.

## Client (Utilisateur)

- **Actions Principales :** Connexion, déconnexion et réinitialisation des informations de connexion ; Consultation de son profil et de ses polices d'assurance ; Soumission d'une demande de suppression de compte ou de données.
- **Contraintes :** Accès strictement limité à ses propres informations.

## III. Entités et Relations (Modèle de Données Conceptuel)

Le système de données reposera sur les entités suivantes et leurs relations.

### 1. Entités

- **Utilisateur (Gestion de l'authentification)**
  - *Champs* : id\_utilisateur (PK), email (Unique), mot\_de\_passe\_hash, role (Admin, Manager, Client).
- **Succursale (Centre de gestion)**
  - *Champs* : id\_succursale (PK), nom, adresse, departement\_associe, id\_manager\_fk (Référence l'Utilisateur responsable).
- **Client (Personne assurée)**
  - *Champs* : id\_client (PK), nom, prenom, date\_naissance, adresse, telephone, id\_succursale\_fk (Référence la Succursale d'enregistrement), id\_utilisateur\_fk (Référence le compte Utilisateur).
- **PoliceAssurance (Contrat)**
  - *Champs* : id\_police (PK), numero\_police (Unique), type\_assurance (Auto, Vie, etc.), date\_debut, date\_fin, prime\_annuelle, id\_client\_fk (Référence le Client concerné).

### 2. Relations Clés

- **Succursale - Client** : Une Succursale enregistre **plusieurs** Clients (1:N).
- **Client - PoliceAssurance** : Un Client possède **plusieurs** Polices (1:N).
- **Utilisateur - Succursale/Client** : L'entité Utilisateur est liée aux entités Succursale (pour les Managers) ou Client (pour les clients finaux) pour gérer les connexions (1:1).

## IV. Mise en Œuvre et Architecture Technique

Le projet sera réalisé en utilisant une architecture web moderne et maintenue, visant la portabilité et une sauvegarde permanente des données.

### 1. Technologies

- **Frontend** : **React** - Framework JavaScript moderne pour une interface utilisateur dynamique et performante.

- **Backend (API) : Node.js (Express)** - Environnement d'exécution rapide et léger pour la construction de l'API RESTful.
- **Base de Données : SQL (PostgreSQL / MySQL)** - Base de données relationnelle pour la conservation structurée des données clients et polices.
- **Hébergement : Cloud (Docker/Kubernetes)** - Solution d'hébergement web pour garantir la disponibilité et la gestion centralisée.

## 2. Architecture

L'application adoptera une architecture **client - serveur** (ou *three-tier*).

1. **Client (Frontend) :** Application React, responsable de l'affichage et des interactions utilisateur.
2. **API (Backend) :** Serveur Express.js, responsable de la logique métier, de l'authentification et de l'autorisation.
3. **Base de Données :** Base de données SQL, responsable de la persistance et de l'intégrité des données.

## 3. Exigences Techniques

- **Sécurité :** Utilisation de l'authentification par jeton (JWT) pour l'API. Hachage des mots de passe.
- **API RESTful :** Mise en place de points de terminaison clairs (/clients, /succursales, /polices) respectant les verbes HTTP (GET, POST, PUT, DELETE).
- **Réactivité :** Le frontend doit être entièrement *responsive* pour s'adapter à toutes les tailles d'écran (mobile, tablette, desktop).